

# Lab 5: Spam Detection

In this assignment, we will build a recurrent neural network to classify a SMS text message as "spam" or "not spam". In the process, you will

1. Clean and process text data for machine learning.
2. Understand and implement a character-level recurrent neural network.
3. Understand batching for a recurrent neural network, and develop custom Dataset and DataLoaders with collate\_fn to implement RNN batching.

## What to submit

Submit a PDF file containing all your code, outputs, and write-up. You can produce a PDF of your Google Colab file by going to File > Print and then save as PDF. The Colab instructions have more information.

Do not submit any other files produced by your code.

Include a link to your colab file in your submission.

## Colab Link

<https://colab.research.google.com/github/sissi820li/Deep-Learning-Project/blob/main/Lab%205/Lab%205%20Spam%20Detection.ipynb>

Include a link to your Colab file here. If you would like the TA to look at your Colab file in case your solutions are cut off, **please make sure that your Colab file is publicly accessible at the time of submission.**

```
In [5]: import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import numpy as np
import time
from torch.nn.utils.rnn import pad_sequence
from torch.utils.data import DataLoader, Dataset
from sklearn.model_selection import train_test_split
```

## Part 1. Data Cleaning [15 pt]

We will be using the "SMS Spam Collection Data Set" available at

<http://archive.ics.uci.edu/ml/datasets/SMS+Spam+Collection>

There is a link to download the "Data Folder" at the very top of the webpage. Download the zip file, unzip it, and upload the file `SMSSpamCollection` to Colab.

## Part (a) [1 pt]

Open up the file in Python, and print out one example of a spam SMS, and one example of a non-spam SMS.

What is the label value for a spam message, and what is the label value for a non-spam message?

The label value for a spam message is "spam" and the label value for a non-spam message is "ham".

```
In [3]: for line in open('SMSSpamCollection'):
        if line[0] == "s":
            print(line)
            break

        for line in open('SMSSpamCollection'):
            if line[0] == 'h':
                print(line)
                break
```

spam Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to receive entry question(std txt rate)T&C's apply 08452810075over18's

ham Go until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...

## Part (b) [1 pt]

How many spam messages are there in the data set? How many non-spam messages are there in the data set?

```
In [4]: spam = 0
        ham = 0
        for line in open('SMSSpamCollection'):
            if line[0] == 's':
                spam += 1
            elif line[0] == 'h':
                ham += 1

        print("Number of spam messages in the data set:", spam)
        print("Number of non-spam messages in the data set:", ham)
```

Number of spam messages in the data set: 747

Number of non-spam messages in the data set: 4827

## Part (c) [4 pt]

load and parse the data into two lists: sequences and labels. Create character-level stoi and itos dictionaries. Reserve the index 0 for padding. Convert the sequences to list of character ids using stoi dictionary and convert the labels to a list of 0s and 1s by assigning class "ham" to 0 and class "spam" to 1.

```
In [8]: sequences = []
labels = []

for line in open('SMSSpamCollection'):
    if line[0] == 'h':
        labels.append('ham')
        sequences.append(line[3:].strip())
    elif line[0] == 's':
        labels.append('spam')
        sequences.append(line[4:].strip())

unique_chars = set(''.join(sequences)) #join all sequences into a single string and
stoi = {'<PAD>': 0}
itos = {0: '<PAD>'}

#Map each unique character to a unique integer ID, starting from 1 (0 is reserved f
for idx, char in enumerate(sorted(unique_chars), start=1):
    stoi[char] = idx
    itos[idx] = char
    #stoi[' '] = 1, stoi['a'] = 2

# Use stoi mapping to convert each char in each sequence
encoded_sequences = [[stoi[char] for char in seq] for seq in sequences]
encoded_labels = [1 if label == 'spam' else 0 for label in labels]

print(f"Total unique characters including padding: {len(stoi)}")
print(f"Original data: '{sequences[2]}'")
print(f"Converted sequence: {encoded_sequences[2]}")
print(f"Encoded label: spam -> {encoded_labels[2]}")
```

Total unique characters including padding: 120

Original data: 'Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to receive entry question(std txt rate)T&C's apply 08452810075over18's'

Converted sequence: [39, 82, 69, 69, 1, 69, 78, 84, 82, 89, 1, 73, 78, 1, 19, 1, 65, 1, 87, 75, 76, 89, 1, 67, 79, 77, 80, 1, 84, 79, 1, 87, 73, 78, 1, 39, 34, 1, 36, 85, 80, 1, 70, 73, 78, 65, 76, 1, 84, 75, 84, 83, 1, 19, 18, 83, 84, 1, 46, 65, 89, 1, 19, 17, 17, 22, 15, 1, 53, 69, 88, 84, 1, 39, 34, 1, 84, 79, 1, 25, 24, 18, 19, 18, 1, 84, 79, 1, 82, 69, 67, 69, 73, 86, 69, 1, 69, 78, 84, 82, 89, 1, 81, 85, 69, 83, 84, 73, 79, 78, 9, 83, 84, 68, 1, 84, 88, 84, 1, 82, 65, 84, 69, 10, 53, 7, 36, 8, 83, 1, 65, 80, 80, 76, 89, 1, 17, 25, 21, 22, 19, 25, 18, 17, 17, 24, 22, 79, 86, 69, 82, 18, 25, 8, 83]

Encoded label: spam -> 1

## Part (d) [4 pt]

Use train\_test\_split function from sklearn ([https://scikit-learn.org/dev/modules/generated/sklearn.model\\_selection.train\\_test\\_split.html](https://scikit-learn.org/dev/modules/generated/sklearn.model_selection.train_test_split.html)) to split the

data indices into `train`, `valid`, and `test`. Use a 60-20-20 split.

You saw in part (b) that there are many more non-spam messages than spam messages. This **imbalance** in our training data will be problematic for training. We can fix this disparity by duplicating spam messages in the training set, so that the training set is roughly balanced.

```
In [9]: x = encoded_sequences
        y = encoded_labels

        indices = list(range(len(x)))

        # Separate 20% for testing and 80% for training and validation
        train_val_idx, test_index = train_test_split(indices, test_size=0.20, random_state=

        # Separate 20% of original data for validation, so take 1/4th of train_val_idx for
        train_index, val_index = train_test_split(train_val_idx, test_size=0.25, random_sta

        # Use indices to grab data for corresponding data sets
        train_x = [x[idx] for idx in train_index]
        train_y = [y[idx] for idx in train_index]

        val_x = [x[idx] for idx in val_index]
        val_y = [y[idx] for idx in val_index]

        test_x = [x[idx] for idx in test_index]
        test_y = [y[idx] for idx in test_index]

        train_spam = []
        for idx, item in enumerate(train_x):
            if train_y[idx] == 1:
                train_spam.append(item)

        # duplicate each spam message 6 more times
        train_x = train_x + train_spam * 6
        train_y = train_y + [1] * (len(train_spam) * 6)

        datasets = {
            "Training Set": train_y,
            "Validation Set": val_y,
            "Test Set": test_y
        }
```

## Part (e) [4 pt]

Since each sequence has a different length, we cannot use the default DataLoader. We need to change the DataLoader such that it can pad different sequence sizes within the batch. To do this, we need to introduce a **collate\_fn** to the DataLoader such that it uses **pad\_sequence** function

([https://pytorch.org/docs/stable/generated/torch.nn.utils.rnn.pad\\_sequence.html](https://pytorch.org/docs/stable/generated/torch.nn.utils.rnn.pad_sequence.html)) to pad the sequences within the batch to the same size.

We also need a custom Dataset class to return a pair of sequence and label for each example. Complete the code below to address these.

Hint:

- <https://stanford.edu/~shervine/blog/pytorch-how-to-generate-data-parallel>
- <https://plainenglish.io/blog/understanding-collate-fn-in-pytorch-f9d1742647d3>

```
In [10]: class MyDataset(Dataset):
    def __init__(self, sequences, labels):
        self.sequences = sequences
        self.labels = labels

    def __len__(self):
        return len(self.sequences)

    def __getitem__(self, idx):
        # Return a single (sequence, label) pair as PyTorch tensors
        seq_tensor = torch.tensor(self.sequences[idx], dtype=torch.long)
        label_tensor = torch.tensor(self.labels[idx], dtype=torch.float32)
        return seq_tensor, label_tensor

    def collate_sequences(batch):
        # Unzip the batch into lists
        sequences = [item[0] for item in batch]
        labels = [item[1] for item in batch]
        padded_sequences = pad_sequence(sequences, batch_first=True, padding_value=0)
        labels_tensor = torch.stack(labels) #stack labels into a single tensor

        return padded_sequences, labels_tensor

train_loader = DataLoader(dataset=MyDataset(train_x, train_y), batch_size=32, shuffle=True)
val_loader = DataLoader(dataset=MyDataset(val_x, val_y), batch_size=32, shuffle=False)
test_loader = DataLoader(dataset=MyDataset(test_x, test_y), batch_size=32, shuffle=False)
```

## Part (f) [1 pt]

Take a look at 10 batches in `train_loader`. What is the maximum length of the input sequence in each batch? How many `<pad>` tokens are used in each of the 10 batches?

```
In [ ]: # For standard DataLoader with shuffle=True, the batch will randomly grab 32 messages
i = 0
for padded_sequences, labels in train_loader:
    if i >= 10:
        break

    pad = 0
    print("Batch", i+1, ":")
    print("\tMaximum Length =", int(padded_sequences.shape[1]))
    for seq in padded_sequences:
        pad += (seq == 0).sum().item()
    print("\tNumber of <pad> tokens =", int(pad))
```

```
i += 1
```

```
Batch 1 :
    Maximum Length = 173
    Number of <pad> tokens = 2268
Batch 2 :
    Maximum Length = 174
    Number of <pad> tokens = 1915
Batch 3 :
    Maximum Length = 180
    Number of <pad> tokens = 2557
Batch 4 :
    Maximum Length = 183
    Number of <pad> tokens = 2628
Batch 5 :
    Maximum Length = 322
    Number of <pad> tokens = 7111
Batch 6 :
    Maximum Length = 163
    Number of <pad> tokens = 1847
Batch 7 :
    Maximum Length = 247
    Number of <pad> tokens = 4961
Batch 8 :
    Maximum Length = 224
    Number of <pad> tokens = 3929
Batch 9 :
    Maximum Length = 257
    Number of <pad> tokens = 5096
Batch 10 :
    Maximum Length = 169
    Number of <pad> tokens = 1524
```

## Part 2. Model Building [8 pt]

Build a recurrent neural network model, using an architecture of your choosing. Use the one-hot embedding of each character as input to your recurrent network. Use one or more fully-connected layers to make the prediction based on your recurrent network output.

Instead of using the RNN output value for the final token, another often used strategy is to max-pool over the entire output array. That is, instead of calling something like:

```
out, _ = self.rnn(x)
self.fc(out[:, -1, :])
```

where `self.rnn` is an `nn.RNN`, `nn.GRU`, or `nn.LSTM` module, and `self.fc` is a fully-connected layer, we use:

```
out, _ = self.rnn(x)
self.fc(torch.max(out, dim=1)[0])
```

This works reasonably in practice. An even better alternative is to concatenate the max-pooling and average-pooling of the RNN outputs:

```
out, _ = self.rnn(x)
out = torch.cat([torch.max(out, dim=1)[0],
                 torch.mean(out, dim=1)], dim=1)
self.fc(out)
```

We encourage you to try out all these options. The way you pool the RNN outputs is one of the "hyperparameters" that you can choose to tune later on.

In [22]: *# You might find this code helpful for obtaining  
# PyTorch one-hot vectors.*

```
ident = torch.eye(10)
print(ident[0]) # one-hot vector
print(ident[1]) # one-hot vector
x = torch.tensor([[1, 2], [3, 4]])
print(ident[x]) # one-hot vectors
```

```
tensor([1., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
tensor([0., 1., 0., 0., 0., 0., 0., 0., 0., 0.])
tensor([[0., 1., 0., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 1., 0., 0., 0., 0., 0., 0., 0.]])

[[0., 0., 0., 1., 0., 0., 0., 0., 0., 0.],
 [0., 0., 0., 0., 1., 0., 0., 0., 0., 0.]])
```

In [11]:

```
class RNN(nn.Module):
    def __init__(self, vocab_size, hidden_size, n_layers=1):
        super(RNN, self).__init__()

        self.ident = torch.eye(vocab_size)
        self.rnn = nn.GRU(vocab_size, hidden_size, n_layers, batch_first=True)
        self.classifier = nn.Linear(hidden_size, 2)

    def forward(self, x):
        inp_x = self.ident[x]
        output, _ = self.rnn(inp_x)

        out = torch.max(output, dim=1)[0]

        out = self.classifier(out)
        return out
```

## Part 3. Training [16 pt]

### Part (a) [4 pt]

Complete the `get_accuracy` function, which will compute the accuracy (rate) of your model across a dataset (e.g. validation set).

```
In [ ]: def get_accuracy(model, data_loader):
        """ Compute the accuracy of the `model` across a dataset `data`

        Example usage:

        >>> model = MyRNN() # to be defined
        >>> get_accuracy(model, valid) # the variable `valid` is from above
        """
        correct = 0
        total = 0

        with torch.no_grad(): # Turn off gradient tracking to save memory and time
            for sequences, labels in data_loader:
                output = model(sequences)
                pred = output.max(1, keepdim=True)[1] # Get pred (0 or 1)
                correct += pred.eq(labels.view_as(pred)).sum().item() # Compare predict
                total += labels.shape[0]

        return correct / total

def evaluate(model, data_loader, criterion):
    """ Evaluate the network on the validation/test set. """
    total_loss = 0.0
    total_err = 0.0
    total_epoch = 0

    with torch.no_grad():

        for sequences, labels in data_loader:
            labels = labels.long()
            outputs = model(sequences)
            loss = criterion(outputs, labels)
            pred = outputs.max(1, keepdim=True)[1]
            corr = pred.ne(labels.view_as(pred))

            total_err += int(corr.sum().item())
            total_loss += loss.item()
            total_epoch += len(labels)

    err = float(total_err) / total_epoch
    avg_loss = float(total_loss) / len(data_loader)

    return err, avg_loss
```

## Part (b) [4 pt]

Train your model. Plot the training curve of your final model. Your training curve should have the training/validation loss and accuracy plotted periodically.

Note: Not all of your batches will have the same batch size. In particular, if your training set does not divide evenly by your batch size, there will be a batch that is smaller than the rest.



```

In [ ]: def get_model_name(name, batch_size, learning_rate, epoch):
        path = "model_{0}_bs{1}_lr{2}_epoch{3}".format(name,
                                                    batch_size,
                                                    learning_rate,
                                                    epoch)

        return path

def train(model, train_loader, valid_loader, num_epochs=30, learning_rate=0.001):
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

    train_err, train_loss, val_err, val_loss = [], [], [], []

    for epoch in range(num_epochs):
        for sequences, labels in train_loader:
            optimizer.zero_grad()
            pred = model(sequences)
            loss = criterion(pred, labels.long()) # Ensure labels are LongTensors
            loss.backward()
            optimizer.step()

        t_err, t_loss = evaluate(model, train_loader, criterion)
        v_err, v_loss = evaluate(model, valid_loader, criterion)

        train_err.append(t_err)
        train_loss.append(t_loss)
        val_err.append(v_err)
        val_loss.append(v_loss)

        print(("Epoch {}: Train err: {}, Train loss: {} | "+
              "Validation err: {}, Validation loss: {}".format(
                  epoch + 1,
                  t_err,
                  t_loss,
                  v_err,
                  v_loss
              )))

        model_name = getattr(model, 'name', 'rnn')
        batch_size = train_loader.batch_size
        model_path = get_model_name(model_name, batch_size, learning_rate, epoch)
        torch.save(model.state_dict(), model_path)

    np.savetxt("{}_train_err.csv".format(model_path), train_err)
    np.savetxt("{}_train_loss.csv".format(model_path), train_loss)
    np.savetxt("{}_val_err.csv".format(model_path), val_err)
    np.savetxt("{}_val_loss.csv".format(model_path), val_loss)

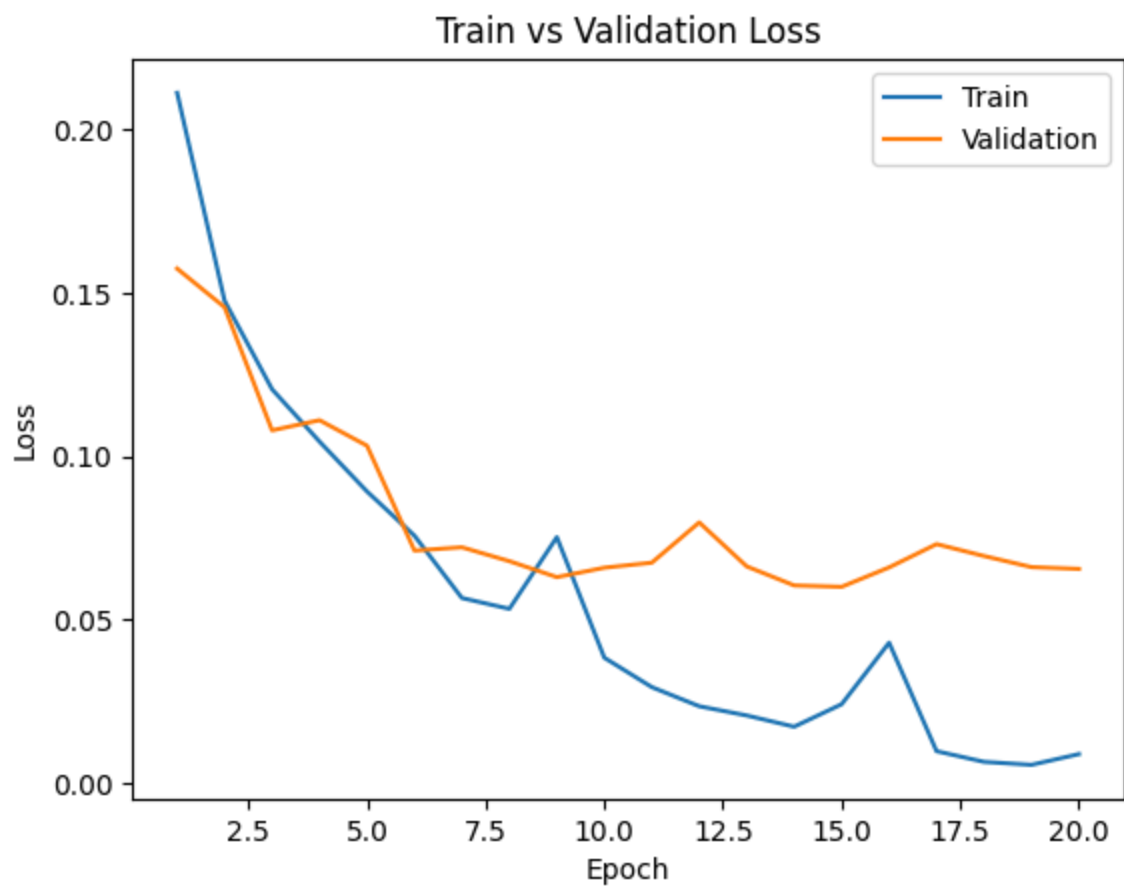
def plot_training_curve(path):
    import matplotlib.pyplot as plt
    train_err = np.loadtxt("{}_train_err.csv".format(path))
    val_err = np.loadtxt("{}_val_err.csv".format(path))
    train_loss = np.loadtxt("{}_train_loss.csv".format(path))
    val_loss = np.loadtxt("{}_val_loss.csv".format(path))
    plt.title("Train vs Validation Error")
    n = len(train_err) # number of epochs

```

```
plt.plot(range(1,n+1), train_err, label="Train")
plt.plot(range(1,n+1), val_err, label="Validation")
plt.xlabel("Epoch")
plt.ylabel("Error")
plt.legend(loc='best')
plt.show()
plt.title("Train vs Validation Loss")
plt.plot(range(1,n+1), train_loss, label="Train")
plt.plot(range(1,n+1), val_loss, label="Validation")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend(loc='best')
plt.show()
```

```
In [ ]: vocab_size = len(stoi) # Number of unique characters including padding
model = RNN(vocab_size=vocab_size, hidden_size=50)
train(model, train_loader, val_loader, learning_rate=0.001, num_epochs = 20)
model_path = get_model_name("rnn", 32, 0.001, 19)
plot_training_curve(model_path)
```

Epoch 1: Train err: 0.05848050649783405, Train loss: 0.21123309554334016 | Validation err: 0.042152466367713005, Validation loss: 0.15741236646260534  
Epoch 2: Train err: 0.051649450183272244, Train loss: 0.14756764182226456 | Validation err: 0.030493273542600896, Validation loss: 0.1455823940890176  
Epoch 3: Train err: 0.04298567144285238, Train loss: 0.12048027547512283 | Validation err: 0.033183856502242155, Validation loss: 0.10788426250219345  
Epoch 4: Train err: 0.036654448517160945, Train loss: 0.10444998733223752 | Validation err: 0.033183856502242155, Validation loss: 0.1109969613807542  
Epoch 5: Train err: 0.02899033655448184, Train loss: 0.08918268891903473 | Validation err: 0.026905829596412557, Validation loss: 0.10318617394992284  
Epoch 6: Train err: 0.023992002665778073, Train loss: 0.07562610097823942 | Validation err: 0.019730941704035873, Validation loss: 0.07107544679726874  
Epoch 7: Train err: 0.01732755748083972, Train loss: 0.05655075148976547 | Validation err: 0.01883408071748879, Validation loss: 0.07208406185465199  
Epoch 8: Train err: 0.016494501832722427, Train loss: 0.05323954240884632 | Validation err: 0.017040358744394617, Validation loss: 0.06783314384520053  
Epoch 9: Train err: 0.02865711429523492, Train loss: 0.07518584112280068 | Validation err: 0.016143497757847534, Validation loss: 0.06293960257566401  
Epoch 10: Train err: 0.013495501499500167, Train loss: 0.0382764517401464 | Validation err: 0.017040358744394617, Validation loss: 0.06583007036575249  
Epoch 11: Train err: 0.007997334221926024, Train loss: 0.02926948385194261 | Validation err: 0.017937219730941704, Validation loss: 0.06736668261832425  
Epoch 12: Train err: 0.006831056314561813, Train loss: 0.023409393650078353 | Validation err: 0.02242152466367713, Validation loss: 0.07969694047101906  
Epoch 13: Train err: 0.007497500833055648, Train loss: 0.020585023936756115 | Validation err: 0.017040358744394617, Validation loss: 0.06623467985274537  
Epoch 14: Train err: 0.004665111629456847, Train loss: 0.017141481992748984 | Validation err: 0.014349775784753363, Validation loss: 0.06040411924145051  
Epoch 15: Train err: 0.01016327890703099, Train loss: 0.024030760089752877 | Validation err: 0.014349775784753363, Validation loss: 0.05997516597355051  
Epoch 16: Train err: 0.0198267244251916, Train loss: 0.042863704866675185 | Validation err: 0.014349775784753363, Validation loss: 0.06592096210058246  
Epoch 17: Train err: 0.000999666777740753, Train loss: 0.009653991943428373 | Validation err: 0.01883408071748879, Validation loss: 0.07305713340507022  
Epoch 18: Train err: 0.000333222592469177, Train loss: 0.006383157023903666 | Validation err: 0.01883408071748879, Validation loss: 0.06940816135944  
Epoch 19: Train err: 0.0, Train loss: 0.0054714001895998544 | Validation err: 0.017937219730941704, Validation loss: 0.06601517329857286  
Epoch 20: Train err: 0.000999666777740753, Train loss: 0.008761703209955235 | Validation err: 0.014349775784753363, Validation loss: 0.0654642181138375



## Part (c) [4 pt]

Choose at least 4 hyperparameters to tune. Explain how you tuned the hyperparameters. You don't need to include your training curve for every model you trained. Instead, explain what hyperparameters you tuned, what the best validation accuracy was, and the reasoning behind the hyperparameter decisions you made.

For this assignment, you should tune more than just your learning rate and epoch. Choose at least 2 hyperparameters that are unrelated to the optimizer.

1. Change the learning rate from 0.001 to 0.0001: A high learning rate can cause the model to take bigger steps and overshoot the optimal solution, so it is good to check if decreasing the learning rate can have better accuracy results. For this model, the best validation accuracy achieved is 94.71%.
2. Change the number of hidden units from 50 to 128: Increasing the number of hidden units allows the model to learn more complex patterns, instead of only learning a few features quickly. The only risk is overfitting, so by increasing the number slightly, it lowers the risk while improving model accuracy. The best validation accuracy achieved is 98.66%.
3. Change the batch size from 32 to 64: DataLoader batch size must be reinitialized to 64 before calling the train function. Increasing the batch size can give the model a more accurate representation on the entire dataset compared to a small batch size of 32. The gradient estimates are more stable and compared to the 32 batch size graph, there is a lot less noise or "jitter". The best validation accuracy achieved is 98.12%.
4. Change pooling strategy from max pooling to max + average pooling: Max pooling has the risk of strictly taking the maximum value and throwing away other information, and can ignore patterns and context of the message. By changing it to max + average pooling, it minimizes that risk since it can capture both key information and surrounding context. The best validation accuracy achieved is 98.56%.

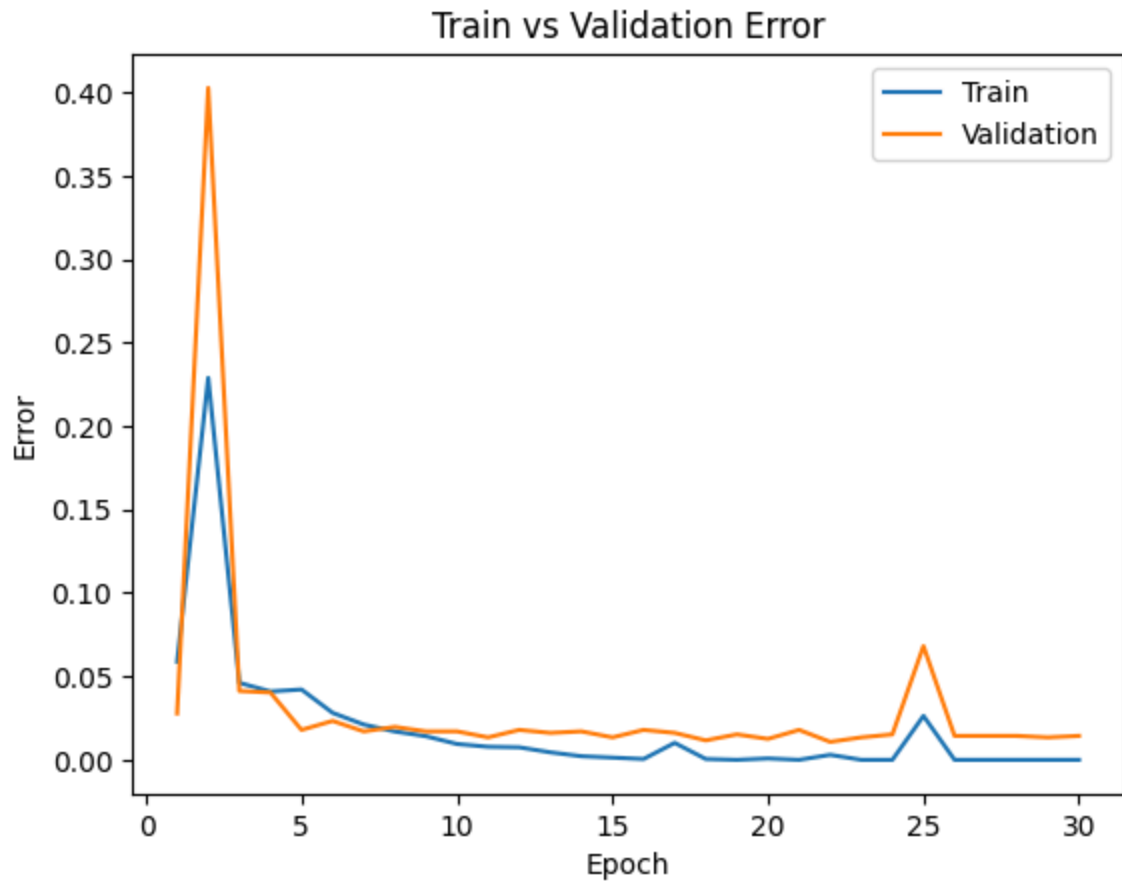
```
In [34]: # Change the Learning rate from 0.001 to 0.0001
model = RNN(vocab_size=vocab_size, hidden_size=50)
train(model, train_loader, val_loader, learning_rate=0.0001, num_epochs = 20)
model_path = get_model_name("rnn", 32, 0.0001, 19)
```

Epoch 1: Train err: 0.4828390536487837, Train loss: 0.675193688970931 | Validation error: 0.8717488789237668, Validation loss: 0.7054981878825597  
 Epoch 2: Train err: 0.17544151949350217, Train loss: 0.5193660520809762 | Validation error: 0.3022421524663677, Validation loss: 0.659533737386976  
 Epoch 3: Train err: 0.06781072975674775, Train loss: 0.34113609442051424 | Validation error: 0.10044843049327354, Validation loss: 0.5239080599376134  
 Epoch 4: Train err: 0.06331222925691436, Train loss: 0.28409728336524454 | Validation error: 0.09417040358744394, Validation loss: 0.32736310277666364  
 Epoch 5: Train err: 0.06097967344218594, Train loss: 0.2417630164467908 | Validation error: 0.08161434977578476, Validation loss: 0.2901127828019006  
 Epoch 6: Train err: 0.06047984005331556, Train loss: 0.22281791222222308 | Validation error: 0.06098654708520179, Validation loss: 0.22370002291032245  
 Epoch 7: Train err: 0.05731422859046984, Train loss: 0.21354623943408754 | Validation error: 0.06367713004484304, Validation loss: 0.21177405544689723  
 Epoch 8: Train err: 0.0546484505164945, Train loss: 0.19952405549268773 | Validation error: 0.05560538116591928, Validation loss: 0.1969844896878515  
 Epoch 9: Train err: 0.05581472842385871, Train loss: 0.19329370792083284 | Validation error: 0.05739910313901345, Validation loss: 0.20160828786236898  
 Epoch 10: Train err: 0.18193935354881707, Train loss: 0.4727551637494818 | Validation error: 0.34618834080717487, Validation loss: 0.8516853102615901  
 Epoch 11: Train err: 0.04981672775741419, Train loss: 0.1772028323104407 | Validation error: 0.05650224215246637, Validation loss: 0.1758329357419695  
 Epoch 12: Train err: 0.07314228590469843, Train loss: 0.21275164531742005 | Validation error: 0.12645739910313902, Validation loss: 0.27224375158548353  
 Epoch 13: Train err: 0.06781072975674775, Train loss: 0.23258459377796092 | Validation error: 0.04753363228699552, Validation loss: 0.15372943271483694  
 Epoch 14: Train err: 0.04965011662779074, Train loss: 0.17518353824840582 | Validation error: 0.06098654708520179, Validation loss: 0.20339352999414717  
 Epoch 15: Train err: 0.04781739420193269, Train loss: 0.17064190772857438 | Validation error: 0.06098654708520179, Validation loss: 0.19457247321094787  
 Epoch 16: Train err: 0.07230923025658113, Train loss: 0.21007225317682357 | Validation error: 0.11569506726457399, Validation loss: 0.3059857689908573  
 Epoch 17: Train err: 0.04748417194268577, Train loss: 0.16425330112589168 | Validation error: 0.059192825112107626, Validation loss: 0.18075744520340647  
 Epoch 18: Train err: 0.046817727424191935, Train loss: 0.16007670229102702 | Validation error: 0.05291479820627803, Validation loss: 0.16619064435362815  
 Epoch 19: Train err: 0.04648450516494502, Train loss: 0.15770256634246796 | Validation error: 0.06457399103139014, Validation loss: 0.19337923271315438  
 Epoch 20: Train err: 0.044151949350216595, Train loss: 0.15250504569725154 | Validation error: 0.05201793721973094, Validation loss: 0.1886267265038831

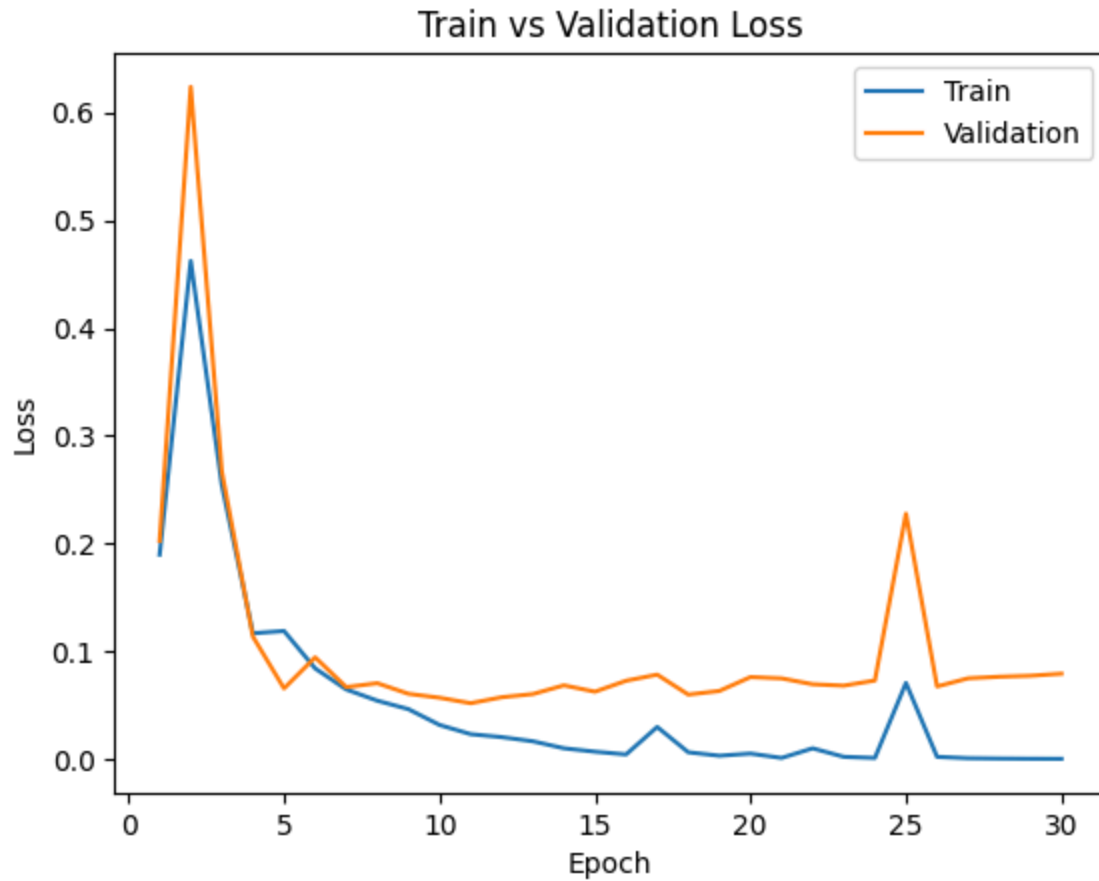
```
In [36]: # Increase the number of hidden units from 50 to 128
model = RNN(vocab_size=vocab_size, hidden_size=128)
train(model, train_loader, val_loader, learning_rate=0.001, num_epochs = 30)
model_path = get_model_name("rnn", 32, 0.001, 29)
plot_training_curve(model_path)
```

Epoch 1: Train err: 0.05881372875708097, Train loss: 0.1897182923127362 | Validation err: 0.02780269058295964, Validation loss: 0.20249391900641578  
Epoch 2: Train err: 0.22892369210263244, Train loss: 0.46228088073908014 | Validation err: 0.4026905829596413, Validation loss: 0.6238307799611773  
Epoch 3: Train err: 0.0461512829056981, Train loss: 0.2542623535115668 | Validation err: 0.04125560538116592, Validation loss: 0.2667825605188097  
Epoch 4: Train err: 0.040986337887370876, Train loss: 0.11674673058726687 | Validation err: 0.04035874439461883, Validation loss: 0.11363707967102528  
Epoch 5: Train err: 0.04215261579473509, Train loss: 0.11887740103052334 | Validation err: 0.017937219730941704, Validation loss: 0.06558032911270857  
Epoch 6: Train err: 0.028157280906364546, Train loss: 0.08418516780031805 | Validation err: 0.023318385650224215, Validation loss: 0.09445922710001468  
Epoch 7: Train err: 0.021159613462179275, Train loss: 0.06465287674020262 | Validation err: 0.017040358744394617, Validation loss: 0.06680503945265498  
Epoch 8: Train err: 0.016994335221592802, Train loss: 0.05420391842663447 | Validation err: 0.019730941704035873, Validation loss: 0.0704435489273497  
Epoch 9: Train err: 0.014161946017994002, Train loss: 0.046349190720456075 | Validation err: 0.017040358744394617, Validation loss: 0.06058741451374122  
Epoch 10: Train err: 0.009496834388537155, Train loss: 0.031692208271434014 | Validation err: 0.017040358744394617, Validation loss: 0.05695395349924053  
Epoch 11: Train err: 0.007830723092302566, Train loss: 0.023142789844720763 | Validation err: 0.013452914798206279, Validation loss: 0.051866196122552666  
Epoch 12: Train err: 0.007497500833055648, Train loss: 0.020392246995780103 | Validation err: 0.017937219730941704, Validation loss: 0.057397049758583306  
Epoch 13: Train err: 0.004498500499833389, Train loss: 0.01654535758326405 | Validation err: 0.016143497757847534, Validation loss: 0.06033723301786397  
Epoch 14: Train err: 0.002165944685104965, Train loss: 0.010049867614338353 | Validation err: 0.017040358744394617, Validation loss: 0.06844638083795351  
Epoch 15: Train err: 0.0013328890369876708, Train loss: 0.006885551383713894 | Validation err: 0.013452914798206279, Validation loss: 0.06255327238551607  
Epoch 16: Train err: 0.0004998333888703765, Train loss: 0.004331300813956137 | Validation err: 0.017937219730941704, Validation loss: 0.07271386565574045  
Epoch 17: Train err: 0.01016327890703099, Train loss: 0.029818850522675116 | Validation err: 0.016143497757847534, Validation loss: 0.07837886382226965  
Epoch 18: Train err: 0.0004998333888703765, Train loss: 0.00631927504288429 | Validation err: 0.011659192825112108, Validation loss: 0.059847538191492536  
Epoch 19: Train err: 0.0, Train loss: 0.0031285273699006907 | Validation err: 0.015246636771300448, Validation loss: 0.06325867243410488  
Epoch 20: Train err: 0.0008330556481172943, Train loss: 0.005122721654883439 | Validation err: 0.012556053811659192, Validation loss: 0.07612088146062368  
Epoch 21: Train err: 0.0, Train loss: 0.0010730741663056952 | Validation err: 0.017937219730941704, Validation loss: 0.0749074631478704  
Epoch 22: Train err: 0.002999000333222259, Train loss: 0.010005621364704913 | Validation err: 0.010762331838565023, Validation loss: 0.06943680559218463  
Epoch 23: Train err: 0.0, Train loss: 0.002110458290073748 | Validation err: 0.013452914798206279, Validation loss: 0.0681659931757687  
Epoch 24: Train err: 0.0, Train loss: 0.001045786295551497 | Validation err: 0.015246636771300448, Validation loss: 0.07294145424509355  
Epoch 25: Train err: 0.026324558480506497, Train loss: 0.0706679339160241 | Validation err: 0.06816143497757847, Validation loss: 0.22774341510874885  
Epoch 26: Train err: 0.0, Train loss: 0.002098573403329399 | Validation err: 0.014349775784753363, Validation loss: 0.0673919200090625  
Epoch 27: Train err: 0.0, Train loss: 0.0008239855272396643 | Validation err: 0.014349775784753363, Validation loss: 0.07497943783528171  
Epoch 28: Train err: 0.0, Train loss: 0.0005176473071785833 | Validation err: 0.014349775784753363, Validation loss: 0.07642146633235306

Epoch 29: Train err: 0.0, Train loss: 0.0003207351033547348 | Validation err: 0.013452914798206279, Validation loss: 0.0773389274290821  
Epoch 30: Train err: 0.0, Train loss: 0.00024610514450846784 | Validation err: 0.014349775784753363, Validation loss: 0.07923288411069994







```
In [37]: # Change batch size from 32 to 64
train_loader = DataLoader(dataset=MyDataset(train_x, train_y), batch_size= 64, shuf
val_loader = DataLoader(dataset=MyDataset(val_x, val_y), batch_size=64, shuffle=Fa

model = RNN(vocab_size=vocab_size, hidden_size=50)
train(model, train_loader, val_loader, learning_rate=0.001, num_epochs = 30)
model_path = get_model_name("rnn", 64, 0.001, 29)
plot_training_curve(model_path)
```

Epoch 1: Train err: 0.08697100966344552, Train loss: 0.27436868497665895 | Validation err: 0.026905829596412557, Validation loss: 0.3105558554331462  
Epoch 2: Train err: 0.04965011662779074, Train loss: 0.17944767330079636 | Validation err: 0.04484304932735426, Validation loss: 0.21269781721962822  
Epoch 3: Train err: 0.04448517160946351, Train loss: 0.15954808979988733 | Validation err: 0.03587443946188341, Validation loss: 0.1477466393262148  
Epoch 4: Train err: 0.040986337887370876, Train loss: 0.1662828137582921 | Validation err: 0.04035874439461883, Validation loss: 0.19821975380182266  
Epoch 5: Train err: 0.04081972675774742, Train loss: 0.12042053016100793 | Validation err: 0.028699551569506727, Validation loss: 0.11812766372329658  
Epoch 6: Train err: 0.03448850383205598, Train loss: 0.10577381210044977 | Validation err: 0.023318385650224215, Validation loss: 0.09725322636465232  
Epoch 7: Train err: 0.028490503165611462, Train loss: 0.09779959851677747 | Validation err: 0.02062780269058296, Validation loss: 0.08497101844598849  
Epoch 8: Train err: 0.0291569476841053, Train loss: 0.08657514183048873 | Validation err: 0.02242152466367713, Validation loss: 0.09337507374584675  
Epoch 9: Train err: 0.028490503165611462, Train loss: 0.07713490942532712 | Validation err: 0.023318385650224215, Validation loss: 0.09626926285111242  
Epoch 10: Train err: 0.02015994668443852, Train loss: 0.06774845999725004 | Validation err: 0.0242152466367713, Validation loss: 0.08234327762491173  
Epoch 11: Train err: 0.016827724091969344, Train loss: 0.05813531740073194 | Validation err: 0.02780269058295964, Validation loss: 0.09189388590554397  
Epoch 12: Train err: 0.015161612795734756, Train loss: 0.04942333554611244 | Validation err: 0.026905829596412557, Validation loss: 0.08580585796799925  
Epoch 13: Train err: 0.017994001999333556, Train loss: 0.05658800741459461 | Validation err: 0.019730941704035873, Validation loss: 0.06673245921006633  
Epoch 14: Train err: 0.016661112962345886, Train loss: 0.05010826947127885 | Validation err: 0.021524663677130046, Validation loss: 0.06752999959927466  
Epoch 15: Train err: 0.008330556481172943, Train loss: 0.036361699037134966 | Validation err: 0.02780269058295964, Validation loss: 0.09026990210016568  
Epoch 16: Train err: 0.00583138953682106, Train loss: 0.02915225660012282 | Validation err: 0.025112107623318385, Validation loss: 0.07806950372954209  
Epoch 17: Train err: 0.010996334555148283, Train loss: 0.03103788474951177 | Validation err: 0.02242152466367713, Validation loss: 0.07686876703519374  
Epoch 18: Train err: 0.007664111962679107, Train loss: 0.02519608042549659 | Validation err: 0.021524663677130046, Validation loss: 0.07886107529823978  
Epoch 19: Train err: 0.009496834388537155, Train loss: 0.02456842647479015 | Validation err: 0.019730941704035873, Validation loss: 0.07624098050615026  
Epoch 20: Train err: 0.004998333888703766, Train loss: 0.01789218061752221 | Validation err: 0.02062780269058296, Validation loss: 0.08042505613735153  
Epoch 21: Train err: 0.01149616794401866, Train loss: 0.025938796751359676 | Validation err: 0.01883408071748879, Validation loss: 0.07473048971345027  
Epoch 22: Train err: 0.010329890036654448, Train loss: 0.022391485835504817 | Validation err: 0.017937219730941704, Validation loss: 0.07558385539515358  
Epoch 23: Train err: 0.0023325558147284237, Train loss: 0.022025778364548658 | Validation err: 0.02062780269058296, Validation loss: 0.0725553773291823  
Epoch 24: Train err: 0.001999333555481506, Train loss: 0.012511564178412106 | Validation err: 0.02242152466367713, Validation loss: 0.08137064713001665  
Epoch 25: Train err: 0.007664111962679107, Train loss: 0.023726502377688488 | Validation err: 0.02780269058295964, Validation loss: 0.10175777309470707  
Epoch 26: Train err: 0.007997334221926024, Train loss: 0.025316862790706628 | Validation err: 0.017040358744394617, Validation loss: 0.07223780496537478  
Epoch 27: Train err: 0.0014995001666111296, Train loss: 0.010198968986514956 | Validation err: 0.01883408071748879, Validation loss: 0.07272007782658976  
Epoch 28: Train err: 0.002499166944351883, Train loss: 0.012760872607109473 | Validation err: 0.021524663677130046, Validation loss: 0.08736353419953957

Epoch 29: Train err: 0.000999666777740753, Train loss: 0.008063217330268248 | Validation err: 0.01883408071748879, Validation loss: 0.07637580237092657  
Epoch 30: Train err: 0.000999666777740753, Train loss: 0.006763975548389507 | Validation err: 0.01883408071748879, Validation loss: 0.07725087575899023





```
In [38]: # Switch from max pooling to max + average pooling in the RNN model
class RNN(nn.Module):
    def __init__(self, vocab_size, hidden_size, n_layers=1):
        super(RNN, self).__init__()
        self.ident = torch.eye(vocab_size)
        self.rnn = nn.GRU(vocab_size, hidden_size, n_layers, batch_first=True)

        self.classifier = nn.Linear(hidden_size * 2, 2) # hidden_size * 2 since con

    def forward(self, x):
        inp_x = self.ident[x]
        output, _ = self.rnn(inp_x)

        out = torch.cat([torch.max(output, dim=1)[0], torch.mean(output, dim=1)], d
        out = self.classifier(out)
        return out

train_loader = DataLoader(dataset=MyDataset(train_x, train_y), batch_size= 32, shuf
val_loader = DataLoader(dataset=MyDataset(val_x, val_y), batch_size=32, shuffle=Fal

model = RNN(vocab_size=vocab_size, hidden_size=50)
train(model, train_loader, val_loader, learning_rate=0.001, num_epochs = 30)
model_path = get_model_name("rnn", 32, 0.001, 29)
plot_training_curve(model_path)
```

Epoch 1: Train err: 0.05681439520159947, Train loss: 0.17783352581070774 | Validation err: 0.03587443946188341, Validation loss: 0.16576905708227838  
Epoch 2: Train err: 0.05198267244251916, Train loss: 0.14231975034790786 | Validation err: 0.02062780269058296, Validation loss: 0.10296957806817122  
Epoch 3: Train err: 0.04448517160946351, Train loss: 0.1238724325109828 | Validation err: 0.02600896860986547, Validation loss: 0.1098071711403983  
Epoch 4: Train err: 0.03898700433188937, Train loss: 0.10822495499110603 | Validation err: 0.019730941704035873, Validation loss: 0.09742181737508093  
Epoch 5: Train err: 0.02865711429523492, Train loss: 0.0852884315994588 | Validation err: 0.02242152466367713, Validation loss: 0.09229612020509584  
Epoch 6: Train err: 0.02465844718427191, Train loss: 0.07843983841148462 | Validation err: 0.02242152466367713, Validation loss: 0.09264927117952279  
Epoch 7: Train err: 0.02149283572142619, Train loss: 0.06333687565502454 | Validation err: 0.01883408071748879, Validation loss: 0.06487149566944156  
Epoch 8: Train err: 0.019660113295568143, Train loss: 0.05804240873994979 | Validation err: 0.019730941704035873, Validation loss: 0.07373255121388606  
Epoch 9: Train err: 0.017494168610463177, Train loss: 0.053548472091317814 | Validation err: 0.021524663677130046, Validation loss: 0.08861558772623539  
Epoch 10: Train err: 0.014161946017994002, Train loss: 0.041553873578680955 | Validation err: 0.017040358744394617, Validation loss: 0.062397164558725696  
Epoch 11: Train err: 0.019326891036321226, Train loss: 0.05150467010853971 | Validation err: 0.016143497757847534, Validation loss: 0.05777424383642418  
Epoch 12: Train err: 0.010496501166277908, Train loss: 0.03436377668688866 | Validation err: 0.019730941704035873, Validation loss: 0.07344762589782476  
Epoch 13: Train err: 0.011829390203265578, Train loss: 0.03265001260560313 | Validation err: 0.015246636771300448, Validation loss: 0.05280207447441561  
Epoch 14: Train err: 0.009996667777407531, Train loss: 0.02660054134445245 | Validation err: 0.014349775784753363, Validation loss: 0.056505761880959786  
Epoch 15: Train err: 0.008163945351549483, Train loss: 0.021630969939461137 | Validation err: 0.014349775784753363, Validation loss: 0.0540509648221944  
Epoch 16: Train err: 0.006331222925691436, Train loss: 0.018082140152902403 | Validation err: 0.013452914798206279, Validation loss: 0.056628612289205196  
Epoch 17: Train err: 0.00583138953682106, Train loss: 0.015309452902193757 | Validation err: 0.017040358744394617, Validation loss: 0.06369524315066104  
Epoch 18: Train err: 0.005498167277574142, Train loss: 0.017517459910719634 | Validation err: 0.017040358744394617, Validation loss: 0.06441128919832409  
Epoch 19: Train err: 0.011829390203265578, Train loss: 0.03462782944521639 | Validation err: 0.030493273542600896, Validation loss: 0.0998664764820465  
Epoch 20: Train err: 0.005998000666444518, Train loss: 0.01408456373982072 | Validation err: 0.015246636771300448, Validation loss: 0.06301937658738876  
Epoch 21: Train err: 0.0038320559813395535, Train loss: 0.0157108106668989 | Validation err: 0.02242152466367713, Validation loss: 0.08176218621499304  
Epoch 22: Train err: 0.0016661112962345886, Train loss: 0.009764235479722513 | Validation err: 0.017040358744394617, Validation loss: 0.06397501588965367  
Epoch 23: Train err: 0.0011662779073642118, Train loss: 0.0069755985066567135 | Validation err: 0.017040358744394617, Validation loss: 0.06803640152460762  
Epoch 24: Train err: 0.009996667777407531, Train loss: 0.027574247532608384 | Validation err: 0.02780269058295964, Validation loss: 0.1156896901716079  
Epoch 25: Train err: 0.0014995001666111296, Train loss: 0.007280237301722778 | Validation err: 0.015246636771300448, Validation loss: 0.06003004694523822  
Epoch 26: Train err: 0.0011662779073642118, Train loss: 0.006187913320513387 | Validation err: 0.017937219730941704, Validation loss: 0.07380503706683937  
Epoch 27: Train err: 0.0004998333888703765, Train loss: 0.004439238453724488 | Validation err: 0.016143497757847534, Validation loss: 0.06839507451222744  
Epoch 28: Train err: 0.0008330556481172943, Train loss: 0.004526081186261937 | Validation err: 0.011659192825112108, Validation loss: 0.06340319877324094

Epoch 29: Train err: 0.00583138953682106, Train loss: 0.020402961557804102 | Validation err: 0.029596412556053813, Validation loss: 0.11539490852238876  
Epoch 30: Train err: 0.0004998333888703765, Train loss: 0.00530117405109427 | Validation err: 0.016143497757847534, Validation loss: 0.06683644853931453





Final Model Parameters:

1. Learning rate: 0.001
2. Batch size: 32
3. Number of hidden units: 128
4. Pooling strategy: Max pooling

Best validation accuracy is 98.48%.

```
In [40]: class bestRNN(nn.Module):
def __init__(self, vocab_size, hidden_size, n_layers=1):
    super(bestRNN, self).__init__()

    self.ident = torch.eye(vocab_size)
    self.rnn = nn.GRU(vocab_size, hidden_size, n_layers, batch_first=True)
    self.classifier = nn.Linear(hidden_size, 2)

def forward(self, x):
    inp_x = self.ident[x]
    output, _ = self.rnn(inp_x)

    out = torch.max(output, dim=1)[0]

    out = self.classifier(out)
    return out
```

```
best_model = bestRNN(vocab_size=vocab_size, hidden_size=128)
train(best_model, train_loader, val_loader, learning_rate=0.001, num_epochs = 30)
bestmodel_path = get_model_name("rnn", 32, 0.001, 29)
plot_training_curve(bestmodel_path)
```



Epoch 1: Train err: 0.05881372875708097, Train loss: 0.19259408635503433 | Validation err: 0.026905829596412557, Validation loss: 0.19750876554421015  
Epoch 2: Train err: 0.07597467510829724, Train loss: 0.2650243876145241 | Validation err: 0.11569506726457399, Validation loss: 0.2901255020073482  
Epoch 3: Train err: 0.044318560479840056, Train loss: 0.13304822110234105 | Validation err: 0.05291479820627803, Validation loss: 0.14033029611621584  
Epoch 4: Train err: 0.036654448517160945, Train loss: 0.10856746437345097 | Validation err: 0.025112107623318385, Validation loss: 0.10743939376303129  
Epoch 5: Train err: 0.02732422525824725, Train loss: 0.07988675011004856 | Validation err: 0.02242152466367713, Validation loss: 0.0911237856639283  
Epoch 6: Train err: 0.016994335221592802, Train loss: 0.05623343056365055 | Validation err: 0.021524663677130046, Validation loss: 0.07132789359561034  
Epoch 7: Train err: 0.013328890369876709, Train loss: 0.03988589303681944 | Validation err: 0.02062780269058296, Validation loss: 0.051479021819042306  
Epoch 8: Train err: 0.016161279573475507, Train loss: 0.04073029482103092 | Validation err: 0.012556053811659192, Validation loss: 0.045622970589569635  
Epoch 9: Train err: 0.005664778407197601, Train loss: 0.022480250828517997 | Validation err: 0.017040358744394617, Validation loss: 0.05150814029454653  
Epoch 10: Train err: 0.0041652782405864714, Train loss: 0.019334538456144187 | Validation err: 0.01883408071748879, Validation loss: 0.05502099668207978  
Epoch 11: Train err: 0.00433188937020993, Train loss: 0.014370733868340625 | Validation err: 0.016143497757847534, Validation loss: 0.04717208620027772  
Epoch 12: Train err: 0.0013328890369876708, Train loss: 0.008344538810791726 | Validation err: 0.01883408071748879, Validation loss: 0.056098586793190665  
Epoch 13: Train err: 0.0004998333888703765, Train loss: 0.006024053546617382 | Validation err: 0.016143497757847534, Validation loss: 0.05633824389328116  
Epoch 14: Train err: 0.0008330556481172943, Train loss: 0.0069835055777381985 | Validation err: 0.017040358744394617, Validation loss: 0.05371960022686315  
Epoch 15: Train err: 0.00016661112962345885, Train loss: 0.002079859490975728 | Validation err: 0.017040358744394617, Validation loss: 0.06028040964405851  
Epoch 16: Train err: 0.0, Train loss: 0.001899137590411788 | Validation err: 0.017937219730941704, Validation loss: 0.06490129128173326  
Epoch 17: Train err: 0.0, Train loss: 0.0012083532086607908 | Validation err: 0.017040358744394617, Validation loss: 0.07241466453249035  
Epoch 18: Train err: 0.0, Train loss: 0.0010241192251650066 | Validation err: 0.016143497757847534, Validation loss: 0.07470595621634857  
Epoch 19: Train err: 0.0, Train loss: 0.000483711091371159 | Validation err: 0.017937219730941704, Validation loss: 0.07524751953148683  
Epoch 20: Train err: 0.0, Train loss: 0.0004321318591927535 | Validation err: 0.017937219730941704, Validation loss: 0.07038515231709296  
Epoch 21: Train err: 0.0, Train loss: 0.0002838952346269736 | Validation err: 0.017040358744394617, Validation loss: 0.08115781591477571  
Epoch 22: Train err: 0.0, Train loss: 0.000206640383726637 | Validation err: 0.017040358744394617, Validation loss: 0.08717200185523584  
Epoch 23: Train err: 0.0, Train loss: 0.0002060884929709031 | Validation err: 0.017937219730941704, Validation loss: 0.08850762675564121  
Epoch 24: Train err: 0.0, Train loss: 0.00013920219578509167 | Validation err: 0.017937219730941704, Validation loss: 0.09312023708579155  
Epoch 25: Train err: 0.0, Train loss: 0.00010331132531315776 | Validation err: 0.017937219730941704, Validation loss: 0.09820875892582989  
Epoch 26: Train err: 0.0, Train loss: 8.041185570047983e-05 | Validation err: 0.017937219730941704, Validation loss: 0.10117686504987172  
Epoch 27: Train err: 0.0, Train loss: 6.507221868356825e-05 | Validation err: 0.017937219730941704, Validation loss: 0.10719786263133366  
Epoch 28: Train err: 0.006997667444185272, Train loss: 0.02114009977618054 | Validation err: 0.017040358744394617, Validation loss: 0.047666557159807

Epoch 29: Train err: 0.0003332222592469177, Train loss: 0.003769547938142576 | Validation err: 0.016143497757847534, Validation loss: 0.05448454218317888  
Epoch 30: Train err: 0.0, Train loss: 0.0013735769994224377 | Validation err: 0.015246636771300448, Validation loss: 0.0623840932152234





### Part (d) [2 pt]

Before we deploy a machine learning model, we usually want to have a better understanding of how our model performs beyond its validation accuracy. An important metric to track is *how well our model performs in certain subsets of the data*.

In particular, what is the model's error rate amongst data with negative labels? This is called the **false positive rate**.

What about the model's error rate amongst data with positive labels? This is called the **false negative rate**.

Report your final model's false positive and false negative rate across the validation set.

```
In [41]: def get_fp_fn_rates(model, data_loader):
    fp = 0 # False positives: predicted spam (1) but the actual label was ham (0)
    tn = 0 # True negatives: predicted ham (0) and the actual label was ham (0)
    fn = 0 # False negatives: predicted ham (0) but the actual label was spam (1)
    tp = 0 # True positives: predicted spam (1) and the actual label was spam (1)

    with torch.no_grad():
        for sequences, labels in data_loader:
            outputs = model(sequences)
            preds = outputs.max(1, keepdim=True)[1].squeeze() # get predicted class
            labels = labels.long()
```

```

fp += ((preds == 1) & (labels == 0)).sum().item()
tn += ((preds == 0) & (labels == 0)).sum().item()

fn += ((preds == 0) & (labels == 1)).sum().item()
tp += ((preds == 1) & (labels == 1)).sum().item()

fpr = fp / (fp + tn) if (fp + tn) > 0 else 0
fnr = fn / (fn + tp) if (fn + tp) > 0 else 0

print(f"False Positive Rate: {fpr:.4f} -> {fp} false positives for {fp + tn} actual hams")
print(f"False Negative Rate: {fnr:.4f} -> {fn} false negatives for {fn + tp} actual spams")

get_fp_fn_rates(best_model, val_loader)

```

False Positive Rate: 0.0082 -> 8 false positives for 972 actual hams

False Negative Rate: 0.0629 -> 9 false negatives for 143 actual spams

## Part (e) [2 pt]

The impact of a false positive vs a false negative can be drastically different. If our spam detection algorithm was deployed on your phone, what is the impact of a false positive on the phone's user? What is the impact of a false negative?

A false negative means that a spam message was able to get through to the user, which would just be annoying at worst. However, a false positive would mean that a real message was labeled as spam, which could prevent the user from ever seeing that message. For example, if the message is urgent and requires user response, a false positive would have a huge impact.

## Part 4. Evaluation [11 pt]

### Part (a) [1 pt]

Report the final test accuracy of your model.

```

In [42]: criterion = nn.CrossEntropyLoss()
test_err, test_loss = evaluate(best_model, test_loader, criterion)
test_accuracy = 1.0 - test_err

print(f"Final Test Accuracy: {test_accuracy * 100:.2f}%")
print(f"Final Test Loss: {test_loss:.4f}")

```

Final Test Accuracy: 98.12%

Final Test Loss: 0.0720

### Part (b) [3 pt]

Report the false positive rate and false negative rate of your model across the test set.

```
In [43]: get_fp_fn_rates(best_model, test_loader)
```

False Positive Rate: 0.0105 -> 10 false positives for 954 actual hams  
False Negative Rate: 0.0683 -> 11 false negatives for 161 actual spams

## Part (c) [3 pt]

What is your model's prediction of the **probability** that the SMS message "machine learning is sooo cool!" is spam?

Hint: To begin, use `stoi` to look up the index of each character in the vocabulary.

```
In [48]: msg = "machine learning is sooo cool!"
indices = [stoi[char] for char in msg] # Convert char to indices using vocab mappin
sequence_tensor = torch.tensor(indices).unsqueeze(0) # convert to pytorch tensor

logits = best_model(sequence_tensor)
probabilities = F.softmax(logits, dim=1)
spam_prob = probabilities[0][1].item()

print(f"Spam probability of message '{msg}': {spam_prob * 100:.2f}%")
```

Spam probability of message 'machine learning is sooo cool!': 0.11%

## Part (d) [4 pt]

Do you think detecting spam is an easy or difficult task?

Since machine learning models are expensive to train and deploy, it is very important to compare our models against baseline models: a simple model that is easy to build and inexpensive to run that we can compare our recurrent neural network model against.

Explain how you might build a simple baseline model. This baseline model can be a simple neural network (with very few weights), a hand-written algorithm, or any other strategy that is easy to build and test.

**Do not actually build a baseline model. Instead, provide instructions on how to build it.**

Detecting spam is a simple task. One way to do it is to use a TF-IDF Vectorizer, which will count how often specific keywords appear in a message. This is the perfect method for detecting spam, since many spam messages rely on eye catching trigger words instead of sentence context/structure. Not only is this effective for detecting spam, it is simple to implement and test against our RNN model.